

MANUAL DE UNITY Y GITHUB

DE NOVATO A AVANZADO

Guía exhaustiva para el desarrollo colaborativo de videojuegos en Debian GNU/Linux con Bash, Unity Hub, Git LFS, UnityYAMLMerge y GameCI

[Git LFS & Assets Binarios](#)[UnityYAMLMerge 3-Way Driver](#)[Visible Meta Files & GUIDs](#)[Multi-Scene & Prefab Isolation](#)[GameCI & GitHub Actions](#)[Catálogo de Incidentes Críticos](#)

Manual de Unity y GitHub: De Novato a Avanzado en Debian Linux

Plataforma: Debian GNU/Linux (11 Bullseye / 12 Bookworm / Ubuntu LTS)

Motor: Unity 2022 LTS / Unity 6 (6000.x)

Shell: Bash / GNU Coreutils

Herramientas: Git 2.40+, Git LFS, GitHub CLI (`gh`), Unity Hub para Linux, UnityYAMLMerge

Índice de Contenidos

1. [Parte I: Fundamentos y Preparación del Entorno Unity en Debian](#)
 - 1.1 [Anatomía de un Proyecto de Unity: Qué se versiona y qué se ignora](#)
 - 1.2 [Instalación de Git, Git LFS y GitHub CLI en Debian](#)
 - 1.3 [Configuración Crítica del Editor de Unity: Visible Meta Files y Force Text](#)
 - 1.4 [La Regla de Oro de los Archivos `.meta` y los GUIDs](#)
 - 1.5 [El Archivo `.gitignore` Oficial y Optimizado para Unity](#)
 - 1.6 [Configuración Exhaustiva de Git LFS con `.gitattributes`](#)
2. [Parte II: Flujo de Trabajo Esencial Diario \(Nivel Novato\)](#)
 - 2.1 [Inicializar y Publicar un Proyecto de Unity en GitHub](#)
 - 2.2 [Clonación Correcta de Proyectos con Git LFS](#)
 - 2.3 [El Ciclo de Trabajo Seguro: Modificar, Probar y Commitear](#)
 - 2.4 [Conventional Commits Aplicados al Desarrollo de Videojuegos](#)
 - 2.5 [Sincronización sin Romper la Cache \(`pull --rebase` \)](#)
3. [Parte III: Estrategias Colaborativas en Unity \(Nivel Intermedio\)](#)
 - 3.1 [Estrategia de Ramas en Equipos de Videojuegos](#)
 - 3.2 [Arquitectura de Escenas Divididas \(Multi-Scene Editing Aditivo\)](#)
 - 3.3 [Aislamiento de Trabajo Mediante Prefabs Anidados y Variantes](#)
 - 3.4 [Configuración de UnityYAMLMerge como Mergetool Semántico](#)
 - 3.5 [Uso de Git Stash y Worktrees sin Desestabilizar Unity](#)
4. [Parte IV: Soluciones por Temas a Conflictos y Edición Concurrente](#)
 - 4.1 [Tema 1: Prevención Arquitectónica de Conflictos en Unity](#)
 - 4.2 [Tema 2: Conflictos en Archivos `.meta` \(GUID Desincronizado\)](#)
 - 4.3 [Tema 3: Conflictos en Scripts C# \(`.cs` \)](#)
 - 4.4 [Tema 4: Conflictos en Escenas \(`.unity` \) y Prefabs \(`.prefab` \) con UnityYAMLMerge](#)

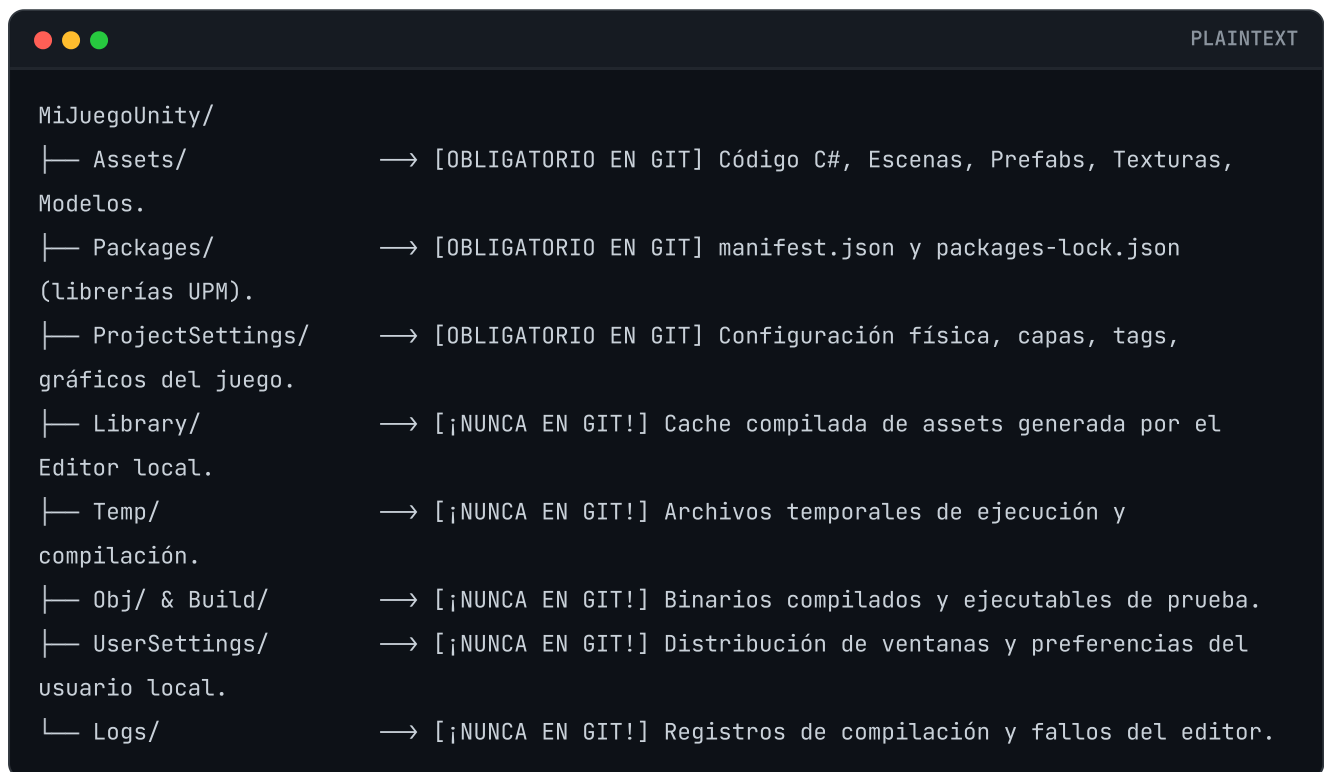
- 4.5 [Tema 5: Forzar una Versión Completa de Asset \(`--ours` vs `--theirs` \)](#)
 - 4.6 [Tema 6: Cambios Locales en el Editor al Hacer Pull](#)
 - 4.7 [Tema 7: Push Rechazado por Desfase y Rebase Seguro con LFS](#)
 - 4.8 [Tema 8: Conflictos en Archivos Binarios y Bloqueo Exclusivo con Git LFS Lock](#)
 - 4.9 [Tema 9: Conflicto de Eliminación de Asset con `.meta` Huérfano](#)
5. [Parte V: Gestión de Paquetes UPM y Dependencias](#)
- 5.1 [Instalación de Paquetes de Unity Mediante URLs de Git](#)
 - 5.2 [Creación y Publicación de Paquetes UPM en Repositorios Privados](#)
6. [Parte VI: Automatización CI/CD con GitHub Actions y Unity \(GameCI\)](#)
- 6.1 [Arquitectura de GameCI para Compilaciones de Videojuegos](#)
 - 6.2 [Activación de Licencias de Unity en GitHub Actions](#)
 - 6.3 [Pipeline Automatizado: Pruebas EditMode/PlayMode y Compilación Linux](#)
 - 6.4 [Subida Automática de Builds a GitHub Releases](#)
7. [Parte VII: Seguridad y Políticas de Repositorio en Equipos de Videojuegos](#)
- 7.1 [Protección de Ramas y Bloqueo de Push sin PR](#)
 - 7.2 [Gestión de Secretos para APIs de Juegos \(Steam, Photon, Firebase\)](#)
 - 7.3 [Gobernanza con `CODEOWNERS` para Artistas y Programadores](#)
8. [Parte VIII: Catálogo Maestro de Incidentes Críticos de Unity en GitHub](#)
- 8.1 [Incidente 1: "Missing Script" Masivo por Desincronización de GUIDs](#)
 - 8.2 [Incidente 2: Escena Corrupta por Edición Manual o Conflicto Mal Resuelto](#)
 - 8.3 [Incidente 3: Subida Accidental de la Carpeta `Library/` \(Repositorio Gigante\)](#)
 - 8.4 [Incidente 4: Repositorio Bloqueado por Superar el Límite de 100 MB](#)
 - 8.5 [Incidente 5: Shaders Magenta / Rosados tras Clonar en Linux](#)
 - 8.6 [Incidente 6: Límite de Ancho de Banda de Git LFS Superado](#)
 - 8.7 [Incidente 7: Desfase de Versiones Menores del Editor de Unity](#)
 - 8.8 [Incidente 8: Archivos Bloqueados por Procesos de Unity al Conmutar Ramas](#)
 - 8.9 [Incidente 9: Fuga de Claves en ScriptableObjects o Archivos de Configuración](#)
 - 8.10 [Incidente 10: Regeneración Limpia y Segura de la Cache Local](#)
-

Parte I: Fundamentos y Preparación del Entorno

Unity en Debian

1.1 Anatomía de un Proyecto de Unity: Qué se versiona y qué se ignora

Un proyecto de Unity contiene miles de archivos autogenerados. Comprender la estructura de carpetas es vital para no saturar tu repositorio ni corromper el motor:

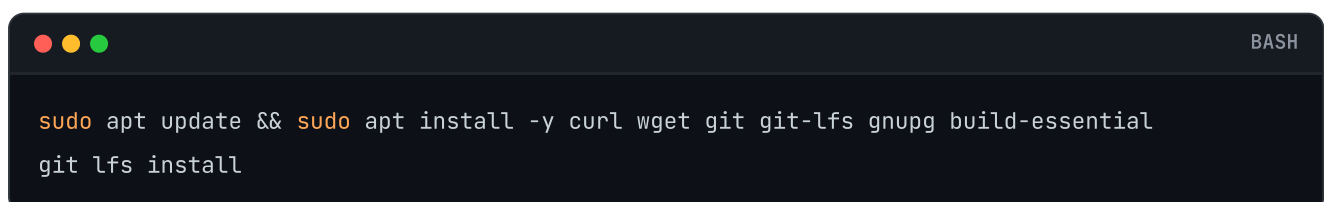


	PLAINTEXT
MiJuegoUnity/	
├─ Assets/	→ [OBLIGATORIO EN GIT] Código C#, Escenas, Prefabs, Texturas, Modelos.
├─ Packages/	→ [OBLIGATORIO EN GIT] manifest.json y packages-lock.json (librerías UPM).
├─ ProjectSettings/	→ [OBLIGATORIO EN GIT] Configuración física, capas, tags, gráficos del juego.
├─ Library/	→ [¡NUNCA EN GIT!] Cache compilada de assets generada por el Editor local.
├─ Temp/	→ [¡NUNCA EN GIT!] Archivos temporales de ejecución y compilación.
├─ Obj/ & Build/	→ [¡NUNCA EN GIT!] Binarios compilados y ejecutables de prueba.
├─ UserSettings/	→ [¡NUNCA EN GIT!] Distribución de ventanas y preferencias del usuario local.
└─ Logs/	→ [¡NUNCA EN GIT!] Registros de compilación y fallos del editor.

1.2 Instalación de Git, Git LFS y GitHub CLI en Debian

En Debian Linux, los proyectos de videojuegos requieren el soporte de Git LFS (Large File Storage) para alojar archivos pesados (texturas 4K, audios sin compresión y modelos 3D).

Paso 1: Instalar dependencias del sistema y Git LFS



```
sudo apt update && sudo apt install -y curl wget git git-lfs gnupg build-essential git lfs install
```

¿Qué hace este comando?

Actualiza los paquetes de Debian, instala Git y el soporte nativo de **Git LFS**, e inicializa los filtros de compresión y punteros (`clean`, `smudge` y `filter`) en la configuración global de tu usuario.

Paso 2: Instalar GitHub CLI (gh)

```
BASH
sudo mkdir -p -m 755 /etc/apt/keyrings
wget -q0- https://cli.github.com/packages/githubcli-archive-keyring.gpg | sudo tee
/etc/apt/keyrings/githubcli-archive-keyring.gpg > /dev/null
sudo chmod go+r /etc/apt/keyrings/githubcli-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/githubcli-archive-
keyring.gpg] https://cli.github.com/packages stable main" | sudo tee
/etc/apt/sources.list.d/github-cli.list > /dev/null
sudo apt update && sudo apt install -y gh
```

¿Qué hace este comando?

Añade el repositorio oficial de GitHub e instala la herramienta `gh` para gestionar repositorios, Pull Requests, issues y autenticación desde la consola.

1.3 Configuración Crítica del Editor de Unity: Visible Meta Files y Force Text

Antes de inicializar Git en cualquier proyecto de Unity, debes verificar dos ajustes obligatorios en el Editor:

1. Abre tu proyecto en Unity en Debian.
2. Navega a **Edit → Project Settings → Version Control**.
 - En **Mode**, selecciona estrictamente: `Visible Meta Files`.
3. Navega a **Edit → Project Settings → Editor**.
 - En **Asset Serialization Mode**, selecciona estrictamente: `Force Text`.

IMPORTANT

`Force Text` obliga a Unity a guardar todas las escenas (`.unity`), prefabs (`.prefab`), materiales (`.mat`) y configuraciones en texto plano **YAML** en lugar de binario propietario. Esto permite ver diferencias (*diffs*) comprensibles y fusionar cambios en Git.

1.4 La Regla de Oro de los Archivos `.meta` y los GUIDs

Por cada archivo y carpeta dentro de `Assets/`, Unity crea un archivo gemelo con extensión `.meta`:

- `Heroe.cs` → `Heroe.cs.meta`
- `Texturas/` → `Texturas.meta`

Dentro de cada `.meta` hay un identificador único global llamado **GUID**:

```
fileFormatVersion: 2
guid: e81b8979d46f4eb2a6886e92f25b2901
```

CAUTION

LA REGLA FUNDAMENTAL:

Si mueves, renombras o borras un asset fuera de Unity (por ejemplo en el explorador de archivos de Debian), **debes mover, renombrar o borrar su archivo `.meta` idénticamente**. Si subes un asset sin su `.meta` a GitHub, Unity en la máquina de tu compañero generará un nuevo GUID aleatorio, rompiendo todas las conexiones de escenas, prefabs y componentes (*Missing Script* y *Missing Prefab*).

1.5 El Archivo `.gitignore` Oficial y Optimizado para Unity

Crea el archivo `.gitignore` en la raíz de tu proyecto de Unity:

```
# =====  
# Archivos autogenerados por Unity (Ignorar)  
# =====  
/[Ll]ibrary/  
/[Tt]emp/  
/[Oo]bj/  
/[Bb]uild/  
/[Bb]uilds/  
/[Ll]ogs/  
/[Uu]ser[Ss]ettings/  
/[Mm]emoryCaptures/  
  
# Asset store packages y respaldos  
/[Aa]ssets/AssetStoreTools*  
  
# Soluciones y proyectos de IDEs (Visual Studio, Rider, VS Code)  
Visual Studio*  
.vs/  
ExportedObj/  
.console/  
*.csproj  
*.unityproj  
*.sln  
*.suo  
*.user  
*.userprefs  
*.pidb  
*.booproj  
*.svd  
*.pdb  
*.mdb  
*.opendb  
*.VC.db  
  
# Compilaciones de Unity  
*.apk  
*.aab  
*.unitypackage  
*.app  
  
# Archivos de SO
```

```
.DS_Store  
Thumbs.db
```

1.6 Configuración Exhaustiva de Git LFS con `.gitattributes`

Git LFS evita que tu repositorio crezca decenas de gigabytes al sustituir archivos binarios gigantes por pequeños punteros de texto.

Crea el archivo `.gitattributes` en la raíz de tu proyecto:

```
# =====
# Configuración de finales de línea
# =====
* text=auto eol=lf
*.cs text diff=csharp
*.cginc text
*.shader text
*.hlsl text
*.compute text
*.json text
*.xml text
*.yaml text
*.yml text

# =====
# Unity YAML Merge (Fusión inteligente de escenas y prefabs)
# =====
*.unity merge=unityyamlmerge eol=lf
*.prefab merge=unityyamlmerge eol=lf
*.asset merge=unityyamlmerge eol=lf
*.mat merge=unityyamlmerge eol=lf

# =====
# Git LFS: Modelos 3D
# =====
*.fbx filter=lfs diff=lfs merge=lfs -text
*.obj filter=lfs diff=lfs merge=lfs -text
*.blend filter=lfs diff=lfs merge=lfs -text
*.max filter=lfs diff=lfs merge=lfs -text
*.dae filter=lfs diff=lfs merge=lfs -text

# =====
# Git LFS: Texturas y Gráficos
# =====
*.png filter=lfs diff=lfs merge=lfs -text
*.jpg filter=lfs diff=lfs merge=lfs -text
*.jpeg filter=lfs diff=lfs merge=lfs -text
*.tga filter=lfs diff=lfs merge=lfs -text
*.psd filter=lfs diff=lfs merge=lfs -text
*.tif filter=lfs diff=lfs merge=lfs -text
*.tiff filter=lfs diff=lfs merge=lfs -text
```

```
*.exr filter=lfs diff=lfs merge=lfs -text
*.hdr filter=lfs diff=lfs merge=lfs -text

# =====
# Git LFS: Audio y Video
# =====
*.wav filter=lfs diff=lfs merge=lfs -text
*.mp3 filter=lfs diff=lfs merge=lfs -text
*.ogg filter=lfs diff=lfs merge=lfs -text
*.flac filter=lfs diff=lfs merge=lfs -text
*.mp4 filter=lfs diff=lfs merge=lfs -text
*.mov filter=lfs diff=lfs merge=lfs -text

# =====
# Git LFS: Binarios compilados y fuentes
# =====
*.dll filter=lfs diff=lfs merge=lfs -text
*.so filter=lfs diff=lfs merge=lfs -text
*.dylib filter=lfs diff=lfs merge=lfs -text
*.ttf filter=lfs diff=lfs merge=lfs -text
*.otf filter=lfs diff=lfs merge=lfs -text
```

Parte II: Flujo de Trabajo Esencial Diario (Nivel Novato)

2.1 Inicializar y Publicar un Proyecto de Unity en GitHub

```
cd ~/UnityProjects/MiVideojuego
git init
git lfs install
git add .gitignore .gitattributes
git commit -m "chore: configurar gitignore y gitattributes para Unity con LFS"
git add Assets/ Packages/ ProjectSettings/
git commit -m "feat: inicializar estructura principal del proyecto Unity"
gh repo create MiVideojuego --public --source=. --remote=origin --push
```

¿Qué hace este comando?

Inicializa el repositorio Git en la carpeta del juego, activa los filtros de Git LFS, commitea primero las reglas de exclusión y atributos, commitea las 3 carpetas obligatorias del motor y publica el repositorio en GitHub con un solo comando.

2.2 Clonación Correcta de Proyectos con Git LFS

Al clonar un proyecto de Unity en otra máquina con Debian:

```
git clone git@github.com:usuario/MiVideojuego.git
cd MiVideojuego
git lfs pull
```

¿Qué hace este comando?

Descarga el repositorio y ejecuta `git lfs pull` para descargar los archivos binarios reales (texturas, sonidos) correspondientes a los punteros de texto.

2.3 El Ciclo de Trabajo Seguro: Modificar, Probar y Commitear

Antes de confirmar cambios en Git:

1. Guarda todas las escenas abiertas en Unity (`Ctrl + S`).
2. Espera a que termine la compilación de scripts en Unity (indicador giratorio en la esquina inferior derecha).
3. Inspecciona qué archivos cambiaron:

```
git status -s
```

¿Qué hace este comando?

Muestra los archivos modificados. Verifica siempre que por cada archivo dentro de `Assets/` se encuentre presente su respectivo archivo `.meta` .

```
# Preparar tanto el código como su archivo meta inseparable
git add Assets/Scripts/PlayerController.cs Assets/Scripts/PlayerController.cs.meta
```

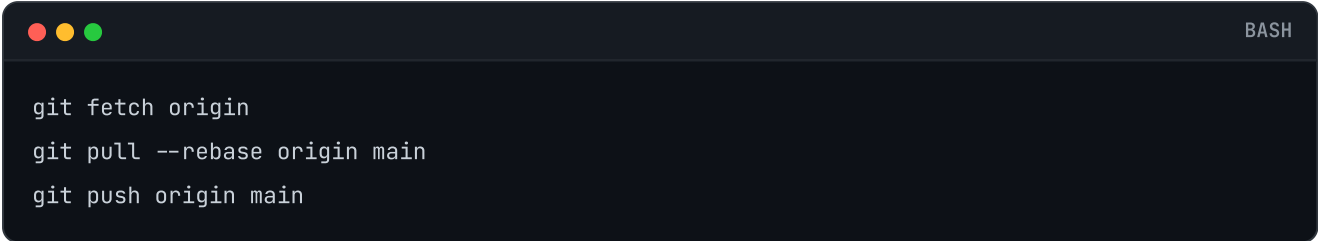
¿Qué hace este comando?

Añade al área de preparación de manera conjunta el script y su identificador GUID.

2.4 Conventional Commits Aplicados al Desarrollo de Videojuegos

Tipo	Uso en Unity	Ejemplo
feat:	Mecánica, escena o sistema nuevo	<code>git commit -m "feat(combat): añadir sistema de combo cuerpo a cuerpo"</code>
fix:	Corrección de bug de física o lógica	<code>git commit -m "fix(physics): corregir salto doble infinito del jugador"</code>
art:	Nuevos modelos, texturas o animaciones	<code>git commit -m "art(boss): importar modelo y texturas de Dragon.fbx"</code>
audio:	Efectos de sonido o bandas sonoras	<code>git commit -m "audio(ui): integrar SFX de clic en menú principal"</code>
perf:	Optimización de draw calls o memoria	<code>git commit -m "perf(rendering): habilitar GPU instancing en materiales"</code>

2.5 Sincronización sin Romper la Cache (`pull --rebase`)



```
git fetch origin
git pull --rebase origin main
git push origin main
```

¿Qué hace este comando?

Descarga las modificaciones de los compañeros y reaplica tus commits locales de manera lineal. Unity detectará los cambios en segundo plano y reimportará únicamente los assets alterados.

Parte III: Estrategias Colaborativas en Unity (Nivel Intermedio)

3.1 Estrategia de Ramas en Equipos de Videojuegos

- `main` / `master` : Versión jugable y estable lista para demostraciones o compilación.
- `develop` : Rama de integración continua donde se prueban las mecánicas unificadas.
- `feature/mecanica-inventario` : Ramas de trabajo aisladas de cada programador o diseñador.

```
git switch -c feature/sistema-inventario
git push -u origin feature/sistema-inventario
```

3.2 Arquitectura de Escenas Divididas (Multi-Scene Editing Aditivo)

El mayor error de los equipos novatos en Unity es trabajar todos sobre una única escena llamada `SampleScene.unity`. Las escenas son archivos YAML de miles de líneas; cuando dos personas mueven objetos al mismo tiempo, resolver el conflicto es extremadamente difícil.

Solución Profesional: Escenas Aditivas

Divide cada nivel del juego en múltiples escenas más pequeñas que se cargan juntas en tiempo de ejecución:

1. `Nivel_01_Core.unity` (GameManager, luces principales, cámaras).
2. `Nivel_01_Entorno.unity` (Terreno, edificios, estática de arte).
3. `Nivel_01_Enemigos.unity` (Generadores de IA, triggers de combate).
4. `Nivel_01_Audio.unity` (Fuentes de sonido de ambiente y música).

De este modo:

- El artista de escenarios edita `Nivel_01_Entorno.unity`.
- El diseñador de combate edita `Nivel_01_Enemigos.unity`.
- **Nunca se produce conflicto porque son archivos físicos distintos en Git.**

3.3 Aislamiento de Trabajo Mediante Prefabs Anidados y Variantes

Convierte cada elemento interactivo (jugadores, puertas, interfaces, trampas) en un **Prefab** (`.prefab`).

- Si necesitas ajustar el comportamiento de un enemigo, abre el modo Prefab aislado en Unity y edita `Enemigo.prefab` .
- **Nunca modifiques la escena completa para cambiar un valor:** edita el Prefab. Al commitear, solo se alterará `Enemigo.prefab` y la escena principal permanecerá intacta.

3.4 Configuración de UnityYAMLMerge como Mergetool Semántico

Unity incluye un solucionador semántico interno para fusionar automáticamente escenas y prefabs.

Localiza el ejecutable en Debian (habitualmente dentro de la instalación de Unity Editor en `~/Unity/Hub/Editor/<version>/Editor/Data/Tools/UnityYAMLMerge` o `/opt/unity/Editor/Data/Tools/UnityYAMLMerge`).

Configúralo en tu archivo global de Git:

```
BASH

UNITY_PATH="$HOME/Unity/Hub/Editor/6000.0.23f1/Editor/Data/Tools/UnityYAMLMerge"

git config --global merge.unityyamlmerge.name "Unity SmartMerge"
git config --global merge.unityyamlmerge.driver "$UNITY_PATH merge -h -p --mode=落后 %0 %A
%B %A"
git config --global merge.unityyamlmerge.recursive binary
```

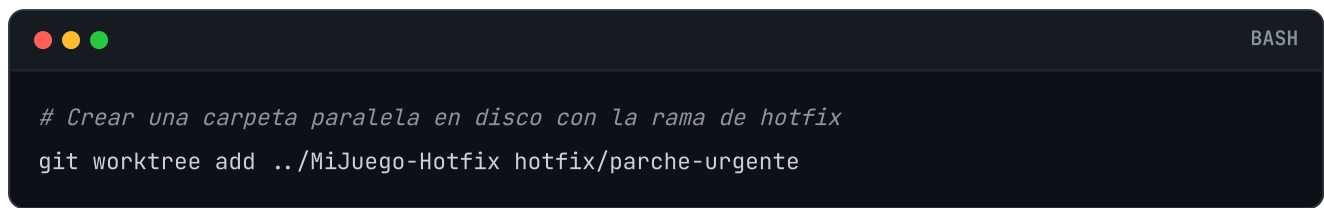
¿Qué hace este comando?

Configura Git para que, ante cualquier colisión en archivos `.unity` o `.prefab` , invoque el analizador inteligente de Unity capaz de entender la jerarquía interna de GameObjects y resolver discrepancias automáticamente.

3.5 Uso de Git Stash y Worktrees sin Desestabilizar Unity

Conmutar de rama abruptamente dentro de Unity provoca que el motor elimine y reimporte gigabytes de cache en `Library/` , congelando el equipo por minutos.

Usar Git Worktrees para trabajar en dos ramas a la vez:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The text "BASH" is in the top right corner. The terminal contains two lines of text: a comment in Spanish and a git command.

```
# Crear una carpeta paralela en disco con la rama de hotfix  
git worktree add ../MiJuego-Hotfix hotfix/parche-urgente
```

¿Qué hace este comando?

Extrae la rama en una carpeta completamente separada. Puedes abrir dos instancias de Unity simultáneas en Debian, una para cada carpeta, sin vaciar la cache de compilación de ninguna de ellas.

Parte IV: Soluciones por Temas a Conflictos y Edición Concurrente

4.1 Tema 1: Prevención Arquitectónica de Conflictos en Unity

1. **Adopta Multi-Scene Editing:** Trabaja en escenas separadas y cárgalas con `SceneManager.LoadSceneAsync("Escena", LoadSceneMode.Additive)`.
2. **Encapsula en Prefabs:** Todo elemento es un prefab; la escena es únicamente un contenedor vacío de instanciación.
3. **Bloqueo de Escenas:** Si un nivel no puede dividirse, utiliza `git lfs lock Niveles/JefeFinal.unity` para avisar al equipo de que esa escena está en edición exclusiva.

4.2 Tema 2: Conflictos en Archivos `.meta` (GUID Desincronizado)

Problema: Dos desarrolladores importaron un asset con el mismo nombre o editaron la configuración de importación de una textura a la vez, produciendo un conflicto en el `.meta`.

```
git status
# Muestra: both modified: Assets/Sprites/Jugador.png.meta
```

Solución:

1. Abre el archivo `.meta` en conflicto con `nano`:

```
nano Assets/Sprites/Jugador.png.meta
```

2. **Conserva siempre un único `guid` homogéneo.** Si el asset ya estaba enlazado en escenas existentes, mantén el GUID original previo para no romper las referencias.
3. Elimina las líneas delimitadoras `<<<<<<`, `=====` y `>>>>>>`.
4. Guarda el archivo y ejecuta:

```
BASH
git add Assets/Sprites/Jugador.png.meta
git rebase --continue
git push origin main
```

4.3 Tema 3: Conflictos en Scripts C# (.cs)

Los scripts C# son texto plano estructurado y se resuelven con el procedimiento estándar de código:

```
BASH
git status
nano Assets/Scripts/Heroe.cs
# Reconciliar la lógica del método y guardar
git add Assets/Scripts/Heroe.cs
git rebase --continue
git push origin main
```

4.4 Tema 4: Conflictos en Escenas (.unity) y Prefabs (.prefab) con UnityYAMLMerge

Si dos personas modificaron la misma escena y ocurre un conflicto:

```
BASH
# 1. Invocar la herramienta inteligente de Unity
git mergetool
```

¿Qué hace este comando?

Lanza `UnityYAMLMerge`. Si los cambios corresponden a `GameObjects` con IDs diferentes en la jerarquía, Unity reconciliará la escena automáticamente sin corromper el árbol YAML.

Si UnityYAMLMerge no logra resolverlo automáticamente (por ejemplo, ambos cambiaron la misma propiedad del mismo componente):

1. Abrirá tu editor gráfico de respaldo (VS Code o Meld).
2. Inspecciona el bloque de IDs en conflicto, decide los valores correctos y guarda.
3. Ejecuta:

```
BASH
git add Assets/Scenes/Nivel1.unity
git commit -m "merge: resolver conflicto en escena Nivel1"
git push origin main
```

4.5 Tema 5: Forzar una Versión Completa de Asset (--ours vs --theirs)

Si una escena o material fue rehecho completamente por uno de los desarrolladores y la otra versión debe descartarse:

```
BASH
# Conservar mi versión completa local:
git checkout --ours Assets/Scenes/Nivel1.unity Assets/Scenes/Nivel1.unity.meta
git add Assets/Scenes/Nivel1.unity Assets/Scenes/Nivel1.unity.meta
git commit -m "resolve: conservar escena local completa"
git push origin main
```

```
BASH
# Aceptar la versión de mi compañero en GitHub íntegramente:
git checkout --theirs Assets/Scenes/Nivel1.unity Assets/Scenes/Nivel1.unity.meta
git add Assets/Scenes/Nivel1.unity Assets/Scenes/Nivel1.unity.meta
git commit -m "resolve: aceptar escena remota del compañero"
git push origin main
```

IMPORTANT

Al forzar una versión con `--ours` o `--theirs`, aplica siempre el comando tanto al archivo del asset como a su archivo `.meta` gemelo para evitar desincronizaciones de identificadores.

4.6 Tema 6: Cambios Locales en el Editor al Hacer Pull

Si hiciste modificaciones en el Inspector de Unity y necesitas descargar cambios nuevos de GitHub:

BASH

```
git stash save "Cambios de inspector en curso"
git pull --rebase origin main
git stash pop
```

¿Qué hace este comando?

Pone a salvo los cambios del inspector, actualiza el proyecto con las novedades remotas y vuelve a volcar tus modificaciones locales.

4.7 Tema 7: Push Rechazado por Desfase y Rebase Seguro con LFS

BASH

```
# Descargar cambios y alinear tus commits por encima
git pull --rebase origin main
# Verificar que los punteros LFS se sincronicen
git lfs push --all origin main
git push origin main
```

4.8 Tema 8: Conflictos en Archivos Binarios y Bloqueo Exclusivo con Git LFS Lock

Los modelos 3D (`.fbx`), texturas (`.tga`) y sonidos no se pueden fusionar.

Bloquear un asset antes de editarlo:

BASH

```
git lfs lock Assets/Modelos/BossFinal.fbx
```

¿Qué hace este comando?

Comunica a GitHub que tienes el archivo bloqueado. Si un compañero intenta hacer push de una modificación sobre `BossFinal.fbx`, GitHub rechazará su push informándole de que tú posees el candado activo.

Consultar y desbloquear al finalizar:

```
BASH

git lfs locks
git lfs unlock Assets/Modelos/BossFinal.fbx
```

4.9 Tema 9: Conflicto de Eliminación de Asset con `.meta` Huérfano

Ocurre cuando un desarrollador borró un asset dentro de Unity y otro añadió un script que dependía de él.

```
BASH

# Si se decide conservar el asset:
git add Assets/Scripts/Herramienta.cs Assets/Scripts/Herramienta.cs.meta
git commit -m "resolve: mantener herramienta y su meta"

# Si se decide confirmar el borrado de ambos:
git rm Assets/Scripts/Herramienta.cs Assets/Scripts/Herramienta.cs.meta
git commit -m "resolve: eliminar script y meta definitivamente"
```

Parte V: Gestión de Paquetes UPM y Dependencias

5.1 Instalación de Paquetes de Unity Mediante URLs de Git

El Unity Package Manager (UPM) permite consumir librerías directamente desde repositorios de GitHub.

Edita el archivo `Packages/manifest.json`:

```
JSON
{
  "dependencies": {
    "com.cysharp.unitask": "https://github.com/Cysharp/UniTask.git?
path=src/UniTask/Assets/Plugins/UniTask#2.5.5",
    "com.github.koki-h.novicov": "https://github.com/koki-h/Novicov.git#v1.2.0"
  }
}
```

¿Qué hace este comando?

Descarga la librería directamente desde el repositorio oficial de GitHub congelada en la etiqueta de versión `#v...`, garantizando builds reproducibles para todo el equipo.

5.2 Creación y Publicación de Paquetes UPM en Repositorios Privados

Puedes modularizar mecánicas compartidas entre diferentes videojuegos creando un repositorio independiente con la estructura de un paquete UPM:

```
PLAINTEXT
MiLibreriaUPM/
├─ package.json          → Manifiesto con "name": "com.miestudio.combate"
├─ package.json.meta
├─ Runtime/              → Código fuente que se ejecuta en el juego
└─ Editor/               → Extensiones del inspector de Unity
```

Parte VI: Automatización CI/CD con GitHub Actions y Unity (GameCI)

6.1 Arquitectura de GameCI para Compilaciones de Videojuegos

GameCI es el estándar de la industria para ejecutar pruebas automatizadas y compilaciones de Unity en GitHub Actions dentro de contenedores Docker oficiales con Unity para Linux.

6.2 Activación de Licencias de Unity en GitHub Actions

1. En tu proyecto de GitHub, acude a **Settings** → **Secrets and variables** → **Actions**.
 2. Añade los siguientes secretos:
 - `UNITY_EMAIL` : Tu correo de Unity ID.
 - `UNITY_PASSWORD` : Tu contraseña de Unity ID.
 - `UNITY_LICENSE` : El contenido del archivo de licencia `.ulf` generado con el workflow de activación de GameCI.
-

6.3 Pipeline Automatizado: Pruebas EditMode/PlayMode y Compilación Linux

Crea el archivo `.github/workflows/unity-linux-ci.yml` :

```

name: Unity Linux Build & Test CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  run-tests:
    name: Ejecutar Pruebas EditMode y PlayMode
    runs-on: ubuntu-latest
    steps:
      - name: Descargar repositorio
        uses: actions/checkout@v4
        with:
          lfs: true

      - name: Cache de Library de Unity
        uses: actions/cache@v4
        with:
          path: Library
          key: Library-Linux-${{ hashFiles('Assets/**', 'Packages/**', 'ProjectSettings/**') }}

      - name: Restaurar claves
        restore-keys: |
          Library-Linux-

      - name: Ejecutar Tests de Unity
        uses: game-ci/unity-test-runner@v4
        env:
          UNITY_LICENSE: ${ secrets.UNITY_LICENSE }
          UNITY_EMAIL: ${ secrets.UNITY_EMAIL }
          UNITY_PASSWORD: ${ secrets.UNITY_PASSWORD }
        with:
          projectPath: .
          testMode: all
          artifactsPath: reports/tests

      - name: Subir resultados de pruebas
        uses: actions/upload-artifact@v4
        if: always()

```

```

    with:
      name: Test-Results
      path: reports/tests

build-linux-player:
  name: Compilar Binario StandaloneLinux64
  needs: run-tests
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
      with:
        lfs: true

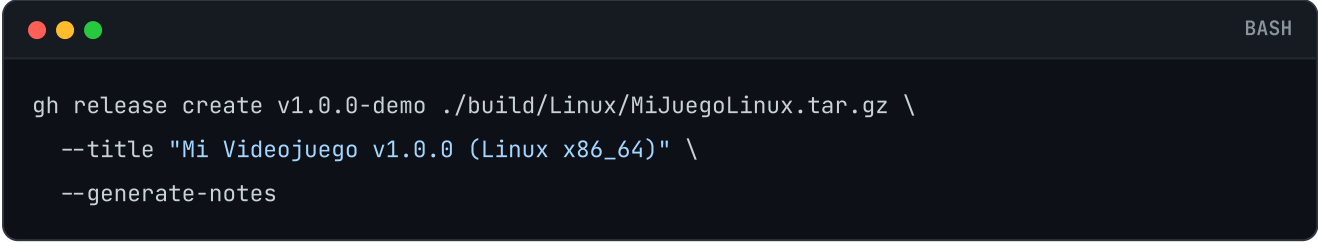
    - uses: actions/cache@v4
      with:
        path: Library
        key: Library-Linux-${{ hashFiles('Assets/**', 'Packages/**', 'ProjectSettings/**') }}

  - name: Compilar Videojuego para Linux
    uses: game-ci/unity-builder@v4
    env:
      UNITY_LICENSE: ${ secrets.UNITY_LICENSE }
      UNITY_EMAIL: ${ secrets.UNITY_EMAIL }
      UNITY_PASSWORD: ${ secrets.UNITY_PASSWORD }
    with:
      projectPath: .
      targetPlatform: StandaloneLinux64
      buildName: MiJuegoLinux
      buildsPath: build/Linux

  - name: Subir artefacto de compilación
    uses: actions/upload-artifact@v4
    with:
      name: MiJuego-Linux-x86_64
      path: build/Linux/StandaloneLinux64

```

6.4 Subida Automática de Builds a GitHub Releases

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text "BASH" is in the top-right corner. The command entered is:

```
gh release create v1.0.0-demo ./build/Linux/MiJuegoLinux.tar.gz \
  --title "Mi Videojuego v1.0.0 (Linux x86_64)" \
  --generate-notes
```

```
gh release create v1.0.0-demo ./build/Linux/MiJuegoLinux.tar.gz \
  --title "Mi Videojuego v1.0.0 (Linux x86_64)" \
  --generate-notes
```

¿Qué hace este comando?

Sube la compilación final del juego generada en el pipeline directamente a la sección de Releases de GitHub para que el equipo de QA pueda descargarla y probarla.

Parte VII: Seguridad y Políticas de Repositorio en Equipos de Videojuegos

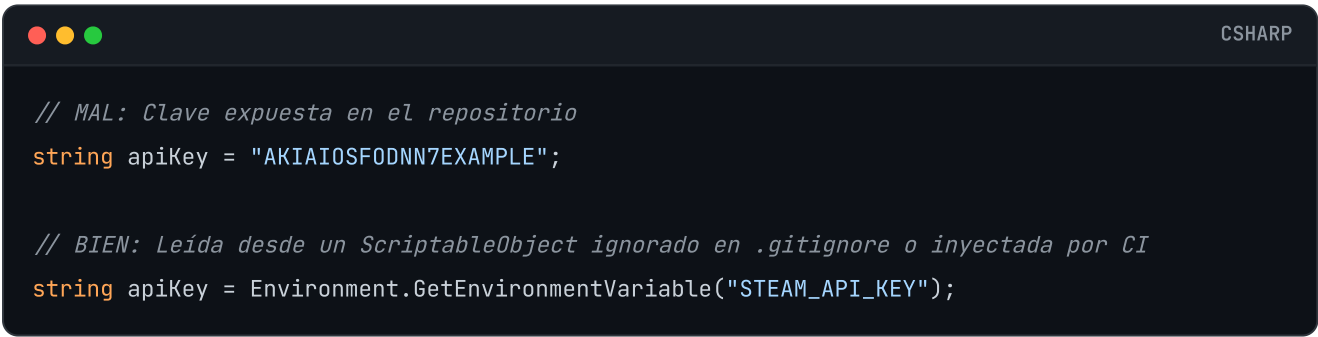
7.1 Protección de Ramas y Bloqueo de Push sin PR

Desde **Settings** → **Rules** → **Rulesets**:

- **Require a pull request before merging:** Bloquea commits directos en `main`.
- **Require status checks to pass:** Exige que los tests automatizados de PlayMode y EditMode concluyan en verde antes del merge.
- **Block force pushes:** Prohíbe el uso de `git push --force` para evitar la pérdida de assets pesados.

7.2 Gestión de Secretos para APIs de Juegos (Steam, Photon, Firebase)

Nunca guardes claves privadas de Steamworks, Photon Engine o servidores multijugador en scripts de C# subidos al repositorio:



```
// MAL: Clave expuesta en el repositorio
string apiKey = "AKIAIOSFODNN7EXAMPLE";

// BIEN: Leída desde un ScriptableObject ignorado en .gitignore o inyectada por CI
string apiKey = Environment.GetEnvironmentVariable("STEAM_API_KEY");
```

7.3 Gobernanza con CODEOWNERS para Artistas y Programadores

Crea `.github/CODEOWNERS` para que los cambios en arte sean revisados por directores de arte y el código por arquitectos de software:

```
# Gobernanza por especialidades
/Assets/Scripts/ @estudio/programadores-core
/Assets/Art/ @estudio/lead-artist
/Assets/Audio/ @estudio/disenador-sonoro
/ProjectSettings/ @estudio/lead-developer
```

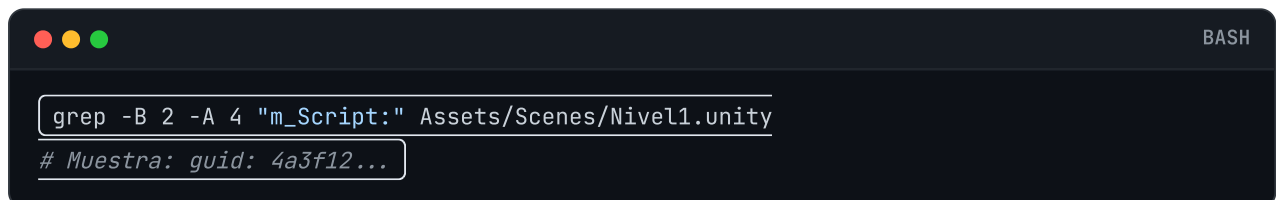
Parte VIII: Catálogo Maestro de Incidentes Críticos de Unity en GitHub

8.1 Incidente 1: "Missing Script" Masivo por Desincronización de GUIDs

Causa: Se subió un script `.cs` sin su correspondiente archivo `.cs.meta`, o alguien cambió manualmente el nombre del archivo fuera de Unity.

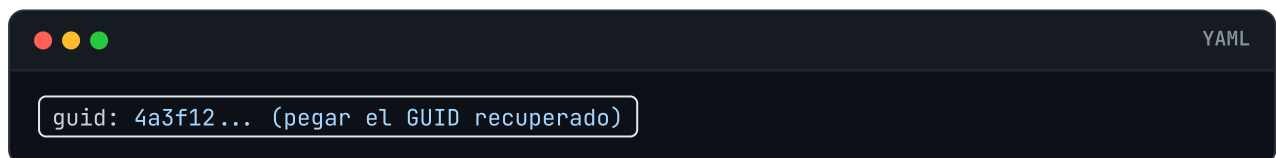
Solución definitiva:

1. Localiza el GUID que la escena o prefab está buscando abriendo la escena con `nano` :



```
BASH
grep -B 2 -A 4 "m_Script:" Assets/Scenes/Nivel1.unity
# Muestra: guid: 4a3f12...
```

2. Abre el archivo `.cs.meta` del script que debería estar asignado y restaura ese GUID exacto:

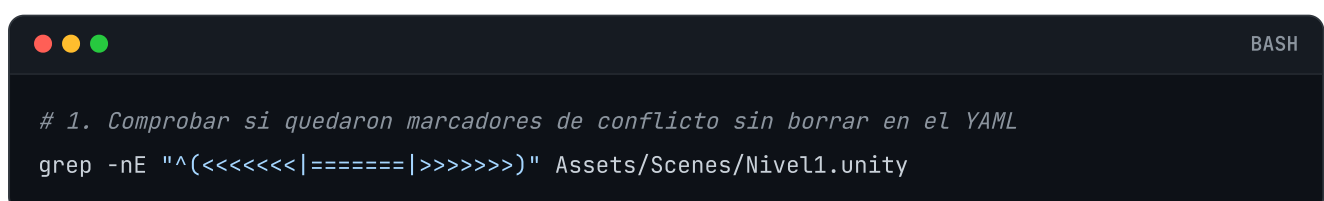


```
YAML
guid: 4a3f12... (pegar el GUID recuperado)
```

3. Guarda el `.meta`, abre Unity y el componente recuperará su script automáticamente sin tener que reasignar variables en todos los GameObjects.

8.2 Incidente 2: Escena Corrupta por Edición Manual o Conflicto Mal Resuelto

Causa: Una escena no abre y Unity arroja el error: `YAML parse error: mapping values are not allowed here`.



```
BASH
# 1. Comprobar si quedaron marcadores de conflicto sin borrar en el YAML
grep -nE "^(<<<<<<|=====>>>>>)" Assets/Scenes/Nivel1.unity
```

¿Qué hace este comando?

Detecta el número de línea exacto donde quedaron restos del conflicto sin limpiar.

Elimina las marcas manuales con `nano`, guarda y Unity cargará la escena con normalidad.

8.3 Incidente 3: Subida Accidental de la Carpeta Library/ (Repositorio Gigante)

Causa: Se inicializó Git sin `.gitignore` y se subieron 20 GB de cache interna de Unity.

```
# 1. Remover Library del índice de Git sin borrar los archivos locales
git rm -r --cached Library/
echo "Library/" >> .gitignore
git commit -m "chore: remover carpeta Library del control de versiones"

# 2. Purgar el historial para recuperar espacio en GitHub
sudo apt install -y git-filter-repo
git filter-repo --path Library --invert-paths --force
git push origin --force --all
```

8.4 Incidente 4: Repositorio Bloqueado por Superar el Límite de 100 MB

Causa: Un modelo 3D o video fue commiteado sin registrar en Git LFS.

```
# Migrar retroactivamente archivos pesados a Git LFS en todo el historial
git lfs migrate import --include="*.fbx,*.mp4,*.psd,*.wav" --everything
git push origin --force --all
```

¿Qué hace este comando?

Convierte automáticamente todos los archivos pesados históricos en punteros Git LFS, reduciendo el repositorio a pocos megabytes y superando el límite de GitHub.

8.5 Incidente 5: Shaders Magenta / Rosados tras Clonar en Linux

Causa: Unity en Linux necesita regenerar la cache de sombreadores para Vulkan u OpenGL.

Solución:

1. En el Editor de Unity en Debian, ve a **Edit → Project Settings → Graphics**.
2. En **Universal Render Pipeline (URP)** o **HDRP**, asegúrate de que el asset de Pipeline esté asignado.
3. Selecciona la carpeta de materiales, haz clic derecho y selecciona **Reimport All**.

8.6 Incidente 6: Límite de Ancho de Banda de Git LFS Superado

Causa: El plan gratuito de GitHub incluye 1 GB de almacenamiento LFS y 1 GB de ancho de banda mensual. Si tu equipo descarga frecuentemente, GitHub bloqueará las descargas de LFS.

Solución inmediata:

- En **Settings → Billing and plans**, compra paquetes de almacenamiento LFS adicionales (\$5/mes por 50 GB), o configura un servidor LFS propio alojado en tu infraestructura con `git-lfs-authenticate`.

8.7 Incidente 7: Desfase de Versiones Menores del Editor de Unity

Causa: Un desarrollador usa Unity `2022.3.10f1` y otro usa `2022.3.25f1`. Unity reescribe metadatos de serialización en cada commit.

Solución:

Crea un archivo `.editorconfig` o `ProjectSettings/ProjectVersion.txt` bloqueado:

```
PLAINTEXT

m_EditorVersion: 2022.3.20f1
m_EditorVersionWithRevision: 2022.3.20f1 (e3215264b312)
```

Todo el equipo debe instalar exactamente la misma versión del editor indicada en `ProjectVersion.txt` a través de Unity Hub.

8.8 Incidente 8: Archivos Bloqueados por Procesos de Unity al Conmutar Ramas

Causa: En Linux, Unity puede mantener descriptores de archivo abiertos en segundo plano.

```
# Cerrar Unity antes de hacer operaciones drásticas de rebase
pkill -f Unity
git switch rama-destino
```

8.9 Incidente 9: Fuga de Claves en ScriptableObjects o Archivos de Configuración

Si se commiteó un archivo `.asset` de configuración con tokens:

```
git filter-repo --path Assets/Settings/ServidorConfig.asset --invert-paths --force
git push origin --force --all
```

8.10 Incidente 10: Regeneración Limpia y Segura de la Cache Local

Si tras hacer `pull` Unity se comporta de manera errática o produce errores de compilación inexplicables:

```
# Cerrar Unity y borrar la cache local corrupta (es 100% seguro)
pkill -f Unity
rm -rf Library/ Temp/ Obj/
```

¿Qué hace este comando?

Al volver a abrir Unity, el motor reconstruirá automáticamente toda la carpeta `Library/` limpia a partir de tus assets de Git, resolviendo cualquier anomalía de cache.